Python Básico

II. Objetos

Objetos simples

string, integer, float, boolean, complex, bytes

Guilherme Ferreira Silveira

gfsilveira@gmail.com

7 de abril de 2026

T7-E04

Benevolent Dictator for Life

Guido van Rossum

1989 -> Python

Um dos focos primordiais de Python era aumentar a produtividade do programador.

Readability => legibilidade

The Zen of Python

Tim Peters

```
import this
```

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

The Zen of Python

Tim Peters

import this

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of [un]ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

Programação Orientada a Objetos (Moldes)

Classes

Abstração:

Instância (nome)

Atributos (características)

Método (ações Classe)

Função (ações Livres)

Herança:

Reuso

Encapsulamento:

Público ou

Privado

Polimorfismo:

Mudanças de comportamento

Programação Orientada a Objetos (Moldes)

Classes

Abstração:

Instância (nome)

Atributos (características)

Método (ações Classe)

Função (ações Livres)

Objetos são instâncias de classes

Variáveis são objetos instanciados.

Uso de variáveis (Instâncias de Objetos)

Python

nome_objeto *recebe* **valor**

```python
# Instanciando objeto do tipo string
lampada = "ligada"
lampada
```

[1] ✓ 0.0s

```
'ligada'
```

nome_objeto.nome_atributo

```python
# Pegando um atributo do objeto string
lampada.__getattribute__
```

[2] ✓ 0.0s

```
<method-wrapper '__getattribute__' of str object at 0x78b6dc0c4570>
```

nome_objeto.nome_método(parâmetro [opcional])

```python
# Usando um método do objeto string
lampada.upper()
```

[3] ✓ 0.0s

```
'LIGADA'
```

Instância - (nome)

Atributos - (características)

Método - (ações Classe)

Função - (ações Livres)

nome_objeto.nome_função(parâmetro [opcional])

```python
# Definindo e suando uma função para desligar a lâmpada
def desligar(lampada: str) -> str:
    lampada = "desligada"
    return lampada

desligar(lampada)
```

[4] ✓ 0.0s

```
'desligada'
```

Escopo em Python

nome_objeto.nome_método(parâmetro [opcional])

```
# Método Local
lampada.count("a")
```

[6] ✓ 0.0s

```
2
```

nome_método(parâmetro [opcional] = {**nome_objeto**})

```
# Método Global
print(lampada)
```

[7] ✓ 0.0s

```
ligada
```

```
# Função Local
desligar(lampada)
```

[8] ✓ 0.0s

```
'desligada'
```

Instância - (nome)

Atributos - (características)

Método - (ações Classe)

Função - (ações Livres)

Local

Global

```
# Método Local
num = 42
num.count("a")
```

[9] ✗ 0.2s

```
---------------------------------------------------------------------------
AttributeError                            Traceback (mo
Cell In[9], line 3
      1 # Método Local
      2 num = 42
----> 3 num.count("a")

AttributeError: 'int' object has no attribute 'count'
```

Uso de variáveis (Instâncias de Objetos)

help()

dir()

Python

```python
# Método help para mostrar a documentação do objeto
help(lampada)
```

[12] ✓ 0.0s

```
No Python documentation found for 'ligada'.
Use help() to get the interactive help utility.
Use help(str) for help on the str class.
```

```python
# Método help para mostrar a documentação do objeto
help(type(lampada))
```

[13] ✓ 0.0s

```
Help on class str in module builtins:

class str(object)
 |  str(object='') -> str
 |  str(bytes_or_buffer[, encoding[, errors]]) -> str
 |
 |  Create a new string object from the given object. If encoding or
 |  errors is specified, then the object must expose a data buffer
 |  that will be decoded using the given encoding and error handler.
 |  Otherwise, returns the result of object.__str__() (if defined)
 |  or repr(object).
 |  encoding defaults to sys.getdefaultencoding().
 |  errors defaults to 'strict'.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
```

```python
# Método dir para mostrar os atributos e métodos disponíveis para o objeto
dir(lampada)
```

[11] ✓ 0.0s

```
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
```

nome_objeto.nome_função(parâmetro [opcional])

```python
# Definindo e suando uma função para desligar a lâmpada
def desligar(lampada: str) -> str:
    lampada = "desligada"
    return lampada

desligar(lampada)
```

[4] ✓ 0.0s

```
'desligada'
```

Nomes de variáveis: objetos, classes e funções

Não podem ser iniciados com números

Não utilizar símbolos, exceto _

Comentários são escritos com #

Não podem ser palavras reservadas

```
False    None    True    and    as
assert    break    class    continue
def    del    elif    else    except
finally    for    from    global    if
import    in    is    lambda    not
nonlocal    or    pass    raise    try
return    while    with    yield
```

Ambiente Integral de Desenvolvimento – IDE

Untitled58.ipynb Salvando...
Arquivo Editar Ver Inserir Ambiente de ex
Comandos + Código + Texto Execut
[1] 1 import os
{} os
{} osgeo
{} osgeo_utils
{} osqp
{} ossaudiodev
{} optparse
[ ] 1 Comece a

module: os
Abrir na guia Ver código-fonte
OS routines for NT or Posix depending on what system we're on.
This exports:
- all functions from posix or nt, e.g. unlink, stat, etc.
- os.path is either posixpath or ntpath
- os.name is either 'posix' or 'nt'

Aju
(function) def dir(
o: object = ...,
/
) -> list[str]
Gerar resumo do Copilot
1
2 dir(lampada)
[11] 0.0s
['__add__',
'__class__',

test.ipynb
test.ipynb > import os
+ Código + Markdown Executar Tudo
import os
{} os
{} ossaudiodev
{} _osx_support
{} optparse

test.ipynb
test.ipynb > import os
+ Código + Markdown Executar Tudo Limpar as Saídas de Todas as Células
import os
(module) os
OS routines for NT or Posix depending on what system we're on.
This exports:
- all functions from posix or nt, e.g. unlink, stat, etc.
- os.path is either posixpath or ntpath
- os.name is either 'posix' or 'nt'
- os.curdir is a string representing the current directory (always '.')
- os.pardir is a string representing the parent directory (always '..')
- os.sep is the (or a most common) pathname separator ('/' or '\')
- os.extsep is the extension separator (always '.')
- os.altsep is the alternate pathname separator (None or '/')
- os.pathsep is the component separator used in $PATH etc

Python

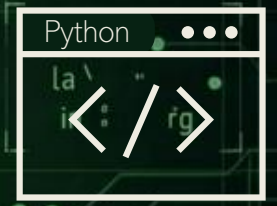
11/25

Conclusões

- Readability
- Beautiful is better than ugly.
- Object-oriented programming (OOP)
- nome_objeto recebe valor
- Escopo
- Nomes de variáveis
- IDE

Vamos praticar?
LIMC-IA

Objetos Simples



Texto: Cadeia de caracteres alpha numéricos **str**: “O valor é R$ 11,00” ‘A luz está “apagada”’

Inteiros: Números inteiros **int**: 42, 0, 1, 25, 3471

Flutuantes: Números decimais **float**: 4.2, 0.34, 225.0, 1.1, 5.0

Booleana: Valores binários (lógicos) **bool:** False, True

Complexo: número complexos (real imaginário) **complex:** 24 + 7j

Bytes: cadeia de caracteres em *ASCII **bytes:** b'\x00\x00\x00\x00\x00'

* American Standard Code for Information Interchange

Vamos exercitar?
LIMC-IA

Exercícios

1 - No paradigma de Programação Orientada a Objetos, o que é um Objeto?

2 - Em Python, o método com nome count() faz parte do objeto da classe string. Este objeto possuí outros atributos e métodos. Qual o código que pode ser usado para verificar se count() é um método também presente no objeto da classe integer?

Exercícios

3 - Marque V para nomes de variáveis válidos e F para nomes inválidos em Python?

( ) nome_objeto

( ) 1_nome _objeto

( ) #nome_objeto

( ) nome1_objeto

( ) lambda

( ) nome_lambda

( ) lambda_nome

( ) @_nome_objeto

( ) Nome objeto

( ) NOME_OBJETO

( ) nome_Objeto

( ) nomeLambda

Exercícios

4 - Para instanciar uma variável que recebe um valor inteiro, com o nome objeto_inteiro, como deve ser o código em Python?

5 - Para instanciar uma variável que recebe um valor decimal, com o nome objeto_decimal, como deve ser o código em Python?

Exercícios

6 - Escreva a classe de objeto ideal para se representar cada uma das seguintes informações:

(a) O número da casa em um endereço

(b) A data de nascimento de uma pessoa

(c) Se uma pessoa é diabética ou não

(d) A altura de uma pessoa em metros

(e) O resultado da divisão de 5 por 2

(f) O resultado da multiplicação de 2,5 por 2

Exercícios

7 - Identifique a classe de objeto em Python dos seguintes valores:

(a) "9 de agosto de 1968"

(b) 1.3

(c) False

(d) -31

(e) “?”

(f) ‘1979’

Exercícios

8 - Instancie variáveis das classes abaixo, recebendo qualquer valor, e retorne os resultados:

string

float

booleano

inteiro

Exercícios

9 - Escreva a classe de objeto ideal para se representar cada uma das seguintes informações:

(a) O nome de um gene;

(b) O *fold change* (razão) de expressão de uma proteína em relação ao controle;

(c) Uma sequencia aminoacídica;

(d) Classificar um gene como *up* ou *down regulated*;

(e) O número de genes classificados como *up* ou *down regulated*

Exercícios

10 - Dado a variável abaixo, qual método pode ser utilizado para buscar o número de ocorrências da sequencia “AUG” presentes no objeto?

sequencia_gene = “ACTGGGUAUGAATTGCGCTTAGACTGGGUAUGAATTGCG”

Python Básico

7 de abril de 2026